

TD n°5 ASDII : Traitements Sur Structures Dynamiques Hiérarchiques

Niveau : 1 LGLSI + 1 LISI

Dans les exercices qui suivent, on suppose disposer de l'implémentation des types et variables suivantes :

Type

Noeud = Enregistrement

Info : type_info (caractère, entier, réel selon le besoin)

FilsG : Pointeur sur Noeud

FilsD : pointeur sur Noeud

Fin Noeud

ArbreB = Pointeur sur Noeud

VAR

F : File de Pointeur sur noeud

Exercice 1

Ecrire l'algorithme de la **Fonction TailleArbre (Racine : ArbreB) : entier** qui retourne la taille d'un arbre binaire (nombre de nœuds).

Exercice 2

Ecrire l'algorithme de la **Fonction MaxAB (R : ArbreB) : Entier** qui permet de déterminer le nœud de valeur maximale dans l'arbre binaire de racine R. On suppose que l'arbre n'est pas vide.

Exercice 3

Ecrire l'algorithme de la **Fonction MinAB (R : ArbreB) : Entier** qui permet de déterminer le nœud de valeur minimale dans l'arbre binaire de racine R. On suppose que l'arbre n'est pas vide.

Exercice 4

Ecrire l'algorithme de la **Fonction Profondeur (R : ArbreB) : Entier** qui permet de déterminer la profondeur de l'arbre de racine R.

Exercice 5

Ecrire l'algorithme de la **Fonction RechercheABR(R : ArbreB; x : entier) : booléen** qui permet de déterminer si l'entier x appartient ou non à l'arbre binaire de recherche de racine R

Exercice 6

Ecrire l'algorithme de la **Fonction MaxABR (R : ArbreB) : Entier** qui permet de déterminer le nœud de valeur maximale dans l'arbre binaire de recherche de racine R On suppose que l'arbre n'est pas vide.

Exercice 7

Ecrire l'algorithme de la **Fonction MinABR (R : ArbreB) : Entier** qui permet de déterminer le nœud de valeur minimale dans l'arbre binaire de recherche de racine R. On suppose que l'arbre n'est pas vide.

Exercice 8

Ecrire l'algorithme de la **fonction EstABR (R : ArbreB) : booléen** qui retourne VRAI si l'arbre de racine R est binaire de recherche, FAUX sinon.

Exercice 9

Ecrire l'algorithme de la **fonction EstPair (R : ArbreB) : booléen** qui retourne VRAI si chaque nœud de l'arbre a soit deux fils soit zéro fils. (FG = FD = NIL ou FG≠Nil ET FD ≠ Nil)

Exercice 10

Ecrire l'algorithme de la **Procédure AjoutNoeud (var R : ArbreB ; NN : ArbreB)** qui permet d'ajouter un nœud NN dans un arbre binaire.

Exercice 11

Ecrire l'algorithme de la **Procédure AjoutNoeudABR (var R : ArbreB; NN : ArbreB)** qui permet d'ajouter un nœud NN dans un arbre binaire **de recherche**.

Exercice 12

Ecrire l'algorithme de la **Procédure CreerABR (R , var RABR : ArbreB)** qui permet de créer un arbre binaire de recherche à partir des nœuds d'un arbre binaire.

Exercice 13

Ecrire l'algorithme de la **procédure SuppNoeud(var R : ArbreB; x : entier)** qui permet de supprimer la valeur x de l'arbre binaire de racine R s'il existe.

Exercice 14

Ecrire l'algorithme de la **procédure SuppNoeudABR(var R : ArbreB; x : entier)** qui permet de supprimer la valeur x de l'arbre binaire de recherche de racine R s'il existe.